

Encrypted Matrix Multiplication Using 3-Dimensional Rotations

Hannah Mahon¹, Shane Kosieradzki²

¹ Georgia Tech Research Institute
`hannah.mahon@gtri.gatech.edu`

² Georgia Institute of Technology
`shanekos1@gmail.com`

Abstract. Fully homomorphic encryption (FHE) enables computations over encrypted data without the need for decryption. Recently there has been an increased interest in developing FHE based algorithms to facilitate encrypted matrix multiplication (EMM) due to rising data security concerns surrounding cyber-physical systems, sensor processing, blockchain, and machine learning. Presently, FHE operations have a high computational overhead, resulting in an increased need for low operational complexity algorithms to compensate. We present a novel matrix encoding and EMM algorithm for power-of-2 cyclotomic based rings, utilizing three-dimensional rotations which offer improvements over the one-dimensional rotations used in previous work. We encode each $d \times d$ matrix as a single, batch-encoded, ciphertext, with minimum ciphertext size d^3 . The proposed algorithm improves the number of plaintext-ciphertext multiplications from $O(d)$ to $O(1)$ and the number of rotations from $O(d)$ to $O(\log_2 d)$. In addition, our work supports rectangular matrix multiplication and matrix packing without incurring additional operations per execution. Benchmarks were obtained with a Microsoft SEAL implementation and compared against leading EMM algorithm, with our work performing 4 times faster for 16×16 matrices on consumer hardware. Our algorithm is compatible with existing encrypted machine learning frameworks and can be a drop-in replacement for existing matrix multiplication algorithms for increased speed. The favorable time complexity is well suited for time sensitive encrypted algorithms such as computer vision, controls, and patient health monitoring.

Keywords: Homomorphic encryption · Secure outsourced matrix computation · Matrix packing.

1 Introduction

1.1 Motivation

Fully homomorphic encryption (FHE) enables computations over encrypted data without the need for decryption. This has significant implications for data privacy in the presence of modern-day cross-party data handling environments.

FHE prevents untrusted third-parties from accessing sensitive data while still maintaining data processing functionality. For example, organizations often out-source data processing tasks for computational power. However, this involves transferring sensitive data to untrusted third-party servers, increasing the risk of data breaches and unauthorized access, in addition to the third party now possessing the raw data for their own usage.

The recent acceleration in machine learning has created an enormous demand for cloud computing providers, such as Microsoft, Amazon, and Google, to perform large computations on private data. FHE is a powerful tool in privacy-protected machine learning to safeguard data within a model such as a neural network and is being researched to improve its practicality in real-life training applications [21] [24] [16]. FHE can also address the growing cyber vulnerabilities, such as man-in-the-middle attacks, present in cyber-physical systems and autonomous technologies including unmanned aerial vehicles and autonomous ground vehicles [30] [26]. It is important to note that while FHE holds much potential for privacy-preserving computations, it is currently magnitudes slower than performing direct computations on unencrypted data. Current research efforts to reduce the high computational expense in FHE include hardware accelerations [15] [1], application-specific algorithmic improvements [21] [24] [30], new FHE schemes [22] [10], and faster encrypted matrix multiplication implementations [17] [13] [28] [7] [31] [11] [23] [29].

1.2 Applications

Encrypted matrix multiplication is emerging as a critical research area due to its applications in cyber-physical systems, sensor processing, blockchain, and machine learning. It is emulated by ciphertext addition, ciphertext rotation, plaintext-ciphertext multiplication, and ciphertext-ciphertext multiplication, where plaintext and ciphertext refer to the unencrypted and encrypted elements of the encryption scheme, respectively. These homomomorphic operations are supported in the widely used FHE schemes [5][8][14][9].

The basic flow for encrypted matrix multiplication, shown in Figure 1, is to encode the matrix into a plaintext, encrypt the plaintext into a ciphertext, perform operations on the ciphertext, decrypt the ciphertext into a plaintext, and finally unencode the plaintext back to a matrix. For example, in the context of a third-party-operated neural net, the user would map their matrix into a holder matrix with the same number of entries as their plaintext size, encode that matrix into a plaintext, encrypt the plaintext into a ciphertext, and then pass the ciphertext to the cloud to operate on. Then the cloud would pass the processed ciphertexts back and the user would decrypt the ciphertext, decode the plaintext, and then unmap the holder matrix to their original matrix dimensions to obtain their data.

Encrypted matrix multiplication has been implemented in real-use applications such as secure multi-party machine learning and blockchain applications, where it enables third party data processing without sharing sensitive data. Researchers have used FHE to securely compute matrix multiplications within

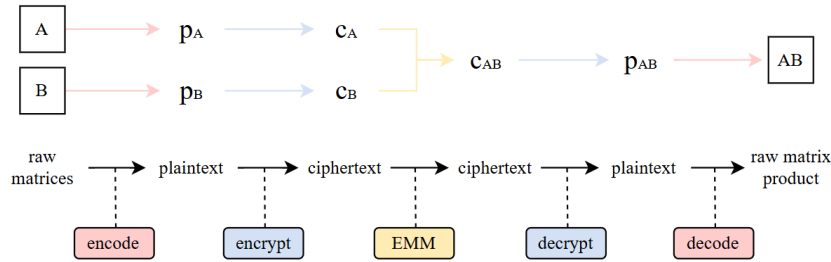


Fig. 1: End-to-end encrypted matrix multiplication workflow: the raw input matrices are encrypted, multiplied in the encrypted domain, and the result is decrypted returning the raw matrix product.

neural networks, protecting both the model’s architecture and training data, and to protect sensitive data in blockchain systems. Jiang et al. [16] developed the framework E2DM implementing an encrypted convolutional neural networks model on encrypted data. This is the first framework which allows for an encrypted model and encrypted data to be sent to a third-party cloud provider to process the data over the model. In their implementation, encrypted square and rectangular matrix multiplications are performed in the homomorphic convolution layer, which are handled by the cloud. Encrypted square matrix multiplication is performed in the first fully connected layer and encrypted rectangular matrix multiplication is performed in the second fully connected layer.

While our work focuses on benchmarking the fundamental performance of encrypted matrix multiplication, the underlying framework of our algorithm is based on the same core set of operations available to any implementation in today’s widely used FHE schemes. This means that our encrypted matrix multiplication algorithm is interchangeable with many existing softwares and can be easily integrated to preexisting implementations to speed up encrypted matrix multiplications. An explicit application of its potential use in a secure machine learning framework is given in Section 3.6 Extensions.

1.3 Literature Review

Significant work on encrypted matrix multiplication has been done in the past decade in reducing the number of homomorphic operations used. The largest bottleneck in encrypted matrix multiplication is the number of ciphertext-ciphertext multiplication operations, followed in order by plaintext-ciphertext multiplication, rotation, and addition operations. There is a concern for space, the number of ciphertexts needed to represent matrices, and size, the amount of data each ciphertext needs to store; however, priority is given to operations, then space, and then size.

Given two square $d \times d$ matrices, a naive matrix multiplication will use d^2 ciphertexts to represent each matrix entry and consume $O(d^3)$ ciphertext-ciphertext multiplications, which is highly inefficient. Halevi et al. [13] developed

a method based on a diagonal encoding of the matrix, which was extended by Lu et al. [23] and Wang et al. [29] to use d ciphertexts and $O(d)$ ciphertext-ciphertext multiplications. Jiang et al. [17] introduce a new matrix encoding, reduce the number of ciphertexts needed to one, and reduce to $O(d)$ multiplication operations. Their algorithm requires a minimum ciphertext size of d^2 slots. Recently, Rizomiliotis et al. [28] leverage Jiang’s encoding, representing each matrix as one ciphertext, to achieve a single ciphertext-ciphertext multiplication. They use $O(d)$ plaintext-ciphertext multiplications and $O(d)$ rotations and require a ciphertext size of at least $2d^3$. The reason their work requires a ciphertext size of $2d^3$ is due to the nature of rotations within power-of-2 cyclotomic polynomial based cipherspaces.

It is significant to note that the previous implementations are supported for ciphertexts of a power-of-2 order. The size of the ciphertext is determined by the initial set up of the ciphertext space. Of the four main publicly available implementations of FHE, all with ciphertext space determined by cyclotomic rings, Microsoft SEAL [6], OpenFHE [4], and PALISADE [27] all specify their implementations are only supported for power-of-2 cyclotomic rings.

Recently, Zheng et al. [31] and Chen et al. [7] proposed algorithms with better performances, however power-of-2 cyclotomic rings are not supported with these methods. Their algorithms use one ciphertext per matrix and one ciphertext-ciphertext multiplication, $O(1)$ plaintext-ciphertext multiplications, and $O(\log d)$ rotations, requiring a minimum ciphertext size of d^3 . In addition, both Zheng et al. and Chen et al. offer efficient algorithms dealing with ciphertexts of smaller size. Their methods use the Chinese Remainder Theorem and are dependent on the cyclotomic ring being isomorphic to smaller cyclotomic rings.

1.4 Our Work

We introduce a novel encrypted matrix multiplication supported in power-of-2 cyclotomic rings, which significantly reduces the number of plaintext-ciphertext multiplications and rotations needed in previous algorithms. Our algorithm consists of two components, highlighted in green in Figure 2:

1. Encoding/decoding: novel encoding and decoding scheme mapping between raw matrices and plaintexts
2. Encrypted matrix multiplication: matrix multiplication algorithm within the ciphertext space operating directly on ciphertexts

Both pieces are necessary for the functionality of the encrypted matrix multiplication algorithm. The encoding and decoding scheme supports batching matrices which means multiple matrices can be mapped to one plaintext to allow for parallel encrypted matrix multiplication in the ciphertext space. The encrypted matrix multiplication algorithm supports multiple uses, as illustrated in Figure 2 by multiplying matrices A, B and D . It also supports matrix batching which means that multiple matrix multiplication executions can occur in parallel with only one application of our encrypted matrix multiplication algorithm.

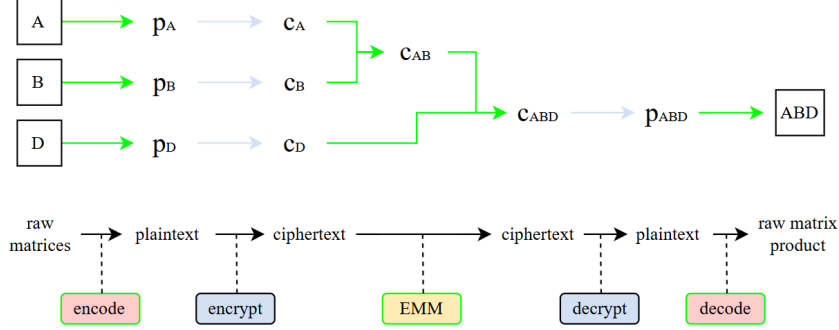


Fig. 2: End-to-end encrypted matrix multiplication workflow with our contributions highlighted in green: the raw input matrices are encrypted, multiplied in the encrypted domain, and the result is decrypted returning the raw matrix product.

The features of our algorithm are as follows. It requires a minimum ciphertext size of d^3 , uses one ciphertext to represent each matrix, one ciphertext-ciphertext multiplication, three plaintext-ciphertext multiplications, and $O(\log d)$ rotations, and has a multiplicative depth of 1 ciphertext-ciphertext multiplication and 3 plaintext-ciphertext multiplications. This tradeoff between an increased multiplicative depth in favor of a constant number of plaintext-ciphertext multiplications scales with larger matrix multiplications. Table 1 summarizes the results for algorithms supported under power-of-2 cyclotomic rings. Our algorithm is based on work by Jiang et al. [16] and Rizomiliotis and Triakosia [28] in minimizing the number of ciphertext-ciphertext multiplications to one. We encode each matrix into a ciphertext to maximize precomputation to minimize the necessary ciphertext-ciphertext multiplications. We are able to reduce the number of plaintext-ciphertext multiplications from $O(d)$ to three and the number of rotation operations from $O(d)$ to $O(\log d)$ by defining a new encoding and new rotation operations to those previously used.

Table 1: Comparison of encrypted $d \times d$ matrix multiplication algorithms supported in BGV and CKKS schemes for power-of-2 cyclotomic rings by number of homomorphisms operations, multiplicative depth, minimum slots per ciphertext, and number of ciphertexts needed to compute the algorithm.

Method	# Mult	# cMult	# Rot	Depth	Min. Dim	# Ciphertexts
Naive	$O(d^3)$	-	-	1 Mult	1	d^2
[13]	$O(d)$	-	$O(d)$	1 Mult	$O(d)$	d
[16]	$O(d)$	$O(d)$	$O(d)$	1 Mult + 2 cMult	$O(d^2)$	1
[28]	1	$O(d)$	$O(d)$	1 Mult + 1 cMult	$2d^3$	1
Our work	1	$O(1)$	$O(\log d)$	1 Mult + 2 cMult	d^3	1

The rest of the paper is structured as follows. The final subsection of the Introduction covers the tradeoffs with our algorithm and similar work. Then Section 2 describes the necessary background for FHE and FHE operations to provide context to the algorithm. Section 3 gives a high-level overview of our encrypted matrix multiplication, then goes into detail for the matrix encoding and 3-dimensional rotations before detailing the algorithm. It concludes with the operation analysis of the algorithm and extensions of the algorithm in matrix packing, rectangular matrix multiplication, and applications in a secure machine-learning framework. Section 4 covers the implementation of the algorithm benchmarked against Rizomiliotis et al.’s algorithm. Section 5 concludes the paper. More mathematical details for the interested reader are given in the Appendix.

1.5 Trade-offs

It is important to talk about the trade-offs with our algorithm compared to the complexity of the proposed alternative methods in Table 1. Our method has equal complexity with respect to lowest number of ciphertext-ciphertext multiplications, and superior complexity with respect to the lowest number of plaintext-ciphertext multiplications and rotations than the next best method [28]. Since our method uses the fewest number of homomorphic operations, we expect it to execute in the least amount of time, as empirically demonstrated in Section 4.

There are two trade-offs. First, in terms of multiplicative depth, our method requires one additional plaintext multiplications than current state-of-the-art algorithms [28]. This means our method will use up more of the “noise budget” of a ciphertext than previous methods, thus requiring a bootstrapping operation earlier to reset the noise. This increase in depth is constant in complexity and does not scale with matrix dimension. The authors believe that the increased noise consumed is worth the price for the speed-ups achieved.

Second, our method requires a minimum of d^3 slots to support $d \times d$ matrices. This cubic spatial complexity presents problems for larger dimensional matrices, as it may be challenging to construct a cryptosystem that has ciphertexts large enough. Furthermore, since the number of available slots in a ciphertext depends on the strength of security parameters used, and the length of FHE operations increases with stronger parameters, large matrices may require security parameters too large for practical computation.

Considering these trade-offs, it is the view of the authors that this method is best suited in practical applications of moderately low dimension. Specifically, it is advantageous where quick short intermittent calculations are required and where the operator’s resource allocation for parallel processing is limited. Problems chosen from the Somewhat Homomorphic Encryption (SHE) research community seem applicable, as SHE prioritizes speed of computation and does not bootstrap thus they must fit all of their computations within the allotted noise budget. Notable examples of SHE research efforts include encrypted robotics

control [19][18] [20], which prioritizes speed above all else. The speed of computation offered by our method is substantial, compared to the modest increase in noise consumed, making our method viable for the SHE problem domain.

2 Preliminaries

2.1 Fully Homomorphic Encryption

Gentry [12] created the groundwork for FHE by developing the first fully homomorphic encryption framework in 2009. Subsequent generations of FHE schemes, second generation BGV [5], third generation TFHE [9], and fourth generation CKKS [8], have built and improved upon his initial methodologies. These schemes are based on the difficulty of the Ring-Learning with Errors (RLWE) problem. BGV and CKKS support SIMD (Single Instruction, Multiple Data) for integer and floating point arithmetic respectively [14][6].

Plaintexts and ciphertexts refer to the unencrypted and encrypted elements in the FHE scheme. The plaintext, containing readable data, is encrypted into a ciphertext where data is unreadable by a third-party. The FHE scheme supports data preserving operations, called homomorphic operations, which can be performed on the ciphertext without the need for decryption. After computations are completed, the ciphertext can be decrypted by the user to recover the resultant plaintext.

The position of the readable data in the plaintext is referred to as the slot. This term is invariant under encryption because when the data is encrypted into the ciphertext, the position within the ciphertext remains the same. An intuitive way to think of a slot is as a container within the plaintext or ciphertext. The size of a plaintext or ciphertext refers to the number of slots it has.

In particular, in SIMD supporting RLWE-based schemes, the plaintext space is typically defined by the polynomial ring $\mathcal{A}_p = \mathbb{Z}_p[x]/(\Phi_m(x))$ where $\Phi_m(x)$ is the m -th cyclotomic polynomial and p is a prime, coprime with m . If $m = 2^k$, then $\Phi_m(x) = x^{m/2} + 1$ and p can be any odd prime. A primitive root of unity $\eta \in \mathbb{Z}_p$ is a zero for $x^m - 1$ and no $x^k - 1$ for $k < m$. A polynomial $f(x) \in \mathcal{A}_p$ is called a plaintext and is constructed by setting $f(x)$ evaluated at $\eta \in \mathbb{Z}_p$, a primitive m th root of unity, to a piece of data. Each primitive m th root of unity corresponds to a slot and a slot can hold one piece of data. This is because f can be evaluated at a primitive m th root of unity with evaluation equal to the data. The variable n is often used to denote the degree of $\Phi_m(x)$ and counts the number of slots in the plaintext. The plaintext $f(x)$ is isomorphic to $(f(\eta^j))_{j \in \mathbb{Z}_m^\times}$ based on an isomorphism between $\mathcal{A}_p$ and the decomposition of $\Phi_m(x)$ in $\mathbb{Z}_p[x]$, see [14] for more details.

An $s \times t$ matrix $A = (a_{i'j'})$ can be encoded as a plaintext by creating a polynomial f which evaluates to entries in A over the primitive m th roots of unity. The term “encoding a matrix” into a plaintext is simply a bijection between entries in the matrix and a subset of the primitive m th roots of unity. The one-to-one correspondence works in the other direction and allows the recovery of

the matrix from the same evaluation points. Lagrange interpolation can be used to define a desired polynomial

$$(f(\eta^j))_{j \in I} = (a_{i'j'})$$

where I is a subset of $\mathbb{Z}_m^\times$ of size st . The encoding defines I and the mappings of $\eta^j \mapsto a_{i'j'}$ for $j \in I$. In practice, it is standard to set $\eta^j \mapsto 0$ for $j \in \mathbb{Z}_m^\times \setminus I$, however any supported values in the scheme may be chosen.

Figure 3 illustrates ciphertexts with their slots linearly organized and uses blue and orange to denote the slots containing irrelevant data and relevant data, respectively. The first ciphertext in a) is an illustration of a ciphertext with 16 slots, with the blue x's representing slots containing no significant data and b) and c) are examples of row-encoded 3×3 and 4×4 matrices into a ciphertext, where each matrix contains the entries $1, \dots, 9$ and $1, \dots, 16$ arranged left to right, top to bottom, respectively.

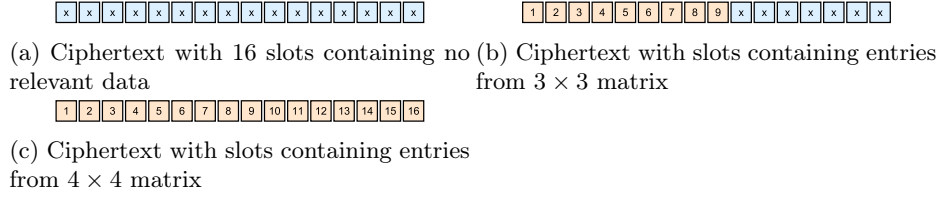


Fig. 3: Examples of matrix data storage in a ciphertext

In RLWE-based FHE schemes, a small amount of error, called noise, must be added to the plaintext during encryption to render it secure, otherwise the ciphertext could be easily reverse engineered back to its plaintext. The noise is generated by a random term sampled from a given distribution and used in the encryption from the plaintext to the ciphertext. Noise is necessary for the security of data, however it comes with a caveat. Homomorphic operations performed on the ciphertext cause the noise terms to accumulate, as when the ciphertext is operated on, the original error terms grows based on the operation. If the accumulation passes a certain threshold, the original plaintext will no longer be recoverable. In FHE this threshold is referred to as a noise budget. FHE schemes have a process called bootstrapping which reduces the noise and resets the noise budget, however it is an extremely computationally expensive process. Thus staying within the noise budget and minimizing operations to avoid bootstrapping is the motivator behind the work done to reduce the number of operations in matrix multiplication algorithms.

2.2 Operations

Most RLWE-based FHE schemes consist of **Encrypt**, **Decrypt**, and **KeyGen** functions which serve as the basic framework to encrypt and decrypt plaintexts

and ciphertexts. The function **KeyGen** generates elements called Galois Keys which are used to define the homomorphic rotation operation. The rotation operation is one of four basic homomorphic operations supported in most RWLE-based FHE schemes, though the particular details for operation implementation vary between schemes.

It is common practice in the FHE field to order the slots in the form of a vector when defining the operations and we will keep to this standard to make explaining the operations clearer. The slots are indexed based on the order of the roots used for evaluation in the corresponding polynomials.

In particular, we have a set of n distinct elements from the field $\mathbb{Z}_m^\times$, denoted as $\{\alpha_1, \alpha_2, \dots, \alpha_n\}$. Each slot in the ciphertext is associated with one of these α_i values. The position of a slot in the vector corresponds to the index i of its associated α_i , so in other words, evaluating the corresponding plaintext polynomial by α_i returns the data housed in the i th slot.

These basic operations are defined as follows, with n number of slots, p_1 a plaintext, and c_1, c_2 ciphertexts. The notation $p_1[i], c_1[i], c_2[i]$ is more succinct and denotes the i th slot of p_1, c_1, c_2 respectively whose values are obtained by evaluating the corresponding polynomial (plaintext or ciphertext) at the corresponding root η^{α_i} .

The addition operation (**Add**(c_1, c_2)) adds c_1 and c_2 slot-wise

$$\mathbf{Add}(c_1, c_2) = (c_1[i] + c_2[i])_{i \in [n]}$$

The rotation operation³ (**Rot**(c_1, j)) rotates or permutes the slots of c_1 j slots to the left

$$\mathbf{Rot}(c_1, j) = (c_1[(i + j) \% n])_{i \in [n]}$$

Using a negative value for j is fine and represents permuting the slots of c_1 $|j|$ slots to the right.

The plaintext-ciphertext multiplication operation (**cMult**(p_1, c_1)) multiplies p_1 and c_1 slot-wise

$$\mathbf{cMult}(p_1, c_1) = (p_1[i] * c_1[i])_{i \in [n]}$$

This is often referred to as a scalar multiplication, although we will continue to use either plaintext-ciphertext multiplication or **cMult** for clarity. The main application for this operation is to mask out elements in the ciphertext. The plaintext contains a one desired slots and zeroes elsewhere, acting as a logical AND.

³ Note: “rotation” is a predefined term for usage in the FHE implementation software. Mathematically, a rotation is a ring automorphism, where more details can be found in the Appendix. The term rotation is often overloaded and is sometimes used to mean *any* type of automorphism. Most FHE research papers discussing the number of operations used in an algorithm will refer to the number of rotation operations, which more accurately means the number of automorphisms. We will follow the lexicon for rotation operations when discussing operation analysis.

The ciphertext-ciphertext multiplication operation ($\mathbf{Mult}(c_1, c_2)$) multiplies c_1 and c_2 slot-wise

$$\mathbf{Mult}(c_1, c_2) = (c_1[i] * c_2[i])_{i \in [n]}$$

This is sometimes referred to as taking the Hadamard product or simply multiplication.

These operations support single instruction multi-data (SIMD) instructions, which is particularly useful in batching. Most RWLE-based FHE schemes allow for matrix batching, also called matrix packing, which describes encoding multiple matrices into one ciphertext. This technique utilizes unused slots in the ciphertext and can reduce the number of operations needed by performing multiple matrix multiplications simultaneously [25] [2] [3]. If a matrix multiplication algorithm supports batching, then it need only be run once over a ciphertext packed with multiple matrices to perform simultaneous multiplications at once. For example, say we want to compute the matrix products $A_i \times B_i$ for $i \in [1, 10]$. Assuming that $A_1, A_2, \dots, A_{10}$ can be packed into a ciphertext and $B_1, B_2, \dots, B_{10}$ can be packed into another ciphertext, the algorithm can intake both ciphertexts and return a ciphertext containing the ten desired products $A_i \times B_i$.

3 Encrypted Matrix Multiplication

3.1 Overview of Encrypted Square Matrix Multiplication Algorithm

In this section we will start by explaining at a high-level our encoding procedure to translate matrices to plaintexts and then explain the three parts forming the encrypted matrix multiplication algorithm. The entire flow is shown in Figure 2 with our contributions highlighted in green.

Since our matrix multiplication algorithm supports batching during the encoding phase, we will explain the encoding process for multiple matrices from an unencrypted state into a plaintext and into a ciphertext. Suppose we have square matrices A_i, B_i for $i \in [1, k]$, where $k \leq nd$ and n is the number of slots in the plaintext and ciphertext. We encode the A_i 's into a plaintext p_A and the B_i 's into a plaintext p_B by first mapping A_i, B_i into (holder/container/intermediate) matrices H_A, H_B of size $n/d \times d$, and then performing a bijection from the entries of H_A, H_B to the slots in p_A, p_B . Slots not containing data from an A_i, B_i will be set to zero in p_A, p_B respectively and will be referred to as zero slots. The ciphertexts are then obtained by $c_A = \mathbf{Encrypt}(p_A), c_B = \mathbf{Encrypt}(p_B)$ and will be the inputs to the matrix multiplication algorithm.

A significant feature of our matrix multiplication algorithm is that it requires one ciphertext-ciphertext multiplication. This is due to a recent technique by Rizomiliotis and Triakosia [28] which reduces the number of ciphertext-ciphertext multiplications to a single operation by utilizing the zero slots of c_A, c_B and filling them with duplicated encrypted values from A_i, B_i . We leverage this in our work by defining ciphertexts $\bar{A}, \bar{B}$ such that we are able to obtain all partial

products needed to compute $A_i \times B_i$ from $\mathbf{Mult}(\bar{A}, \bar{B})$. Then we sum over these partial products and shift the resulting values into a ciphertext $\bar{C}$ which can be unencrypted and decoded by reversing the process above to obtain all values $A_i \times B_i$.

We define automorphisms that are able to compute $\bar{A}, \bar{B}, \bar{C}$ more efficiently than with the traditional left/right rotations. The slots in the ciphertext can be oriented to represent the ciphertext as a 3-dimensional tensor, with automorphisms that mimic 3-dimensional rotations. More precisely, the slots can be arranged to form a $d \times d \times n/d^2$ tensor with $\frac{n}{d^2}$ squares, d rows per square, and d columns per square. We choose an ordering such that there exist automorphisms which will permute the rows, columns, and squares respectively, while preserving the necessary structure of the slots needed for the algorithm row-wise, column-wise, or square-wise. For instance, a row permutation cyclically shifts rows, keeping slots within their original squares and columns.

The automorphisms we define for column and square permutations are not perfect because they perturb some of the slots, however they are sufficient for the matrix multiplication algorithm. While the column and square permutations we define perturb some slots, these perturbations are either masked out or filled with zeros, and thus do not hinder the accuracy of the matrix multiplication algorithm.

We will use the terms “swap rows”, “swap columns”, and “swap squares” to avoid overloading the word rotation and will define them formally in Section 3.3. For the time being, we will provide illustrations to give an intuition for the function of these swap functions and the resulting transformation of the ciphertext. It is easier to diagram two-dimensional rectangles, so we will flatten our cube into a rectangle by stacking each slice of the cube sequentially, with the front-facing square positioned at the bottom of the rectangle. The formal definitions for each will be provided later on.

Our matrix multiplication algorithm is shown in Figure 4 and Figure 5 for the product $A \times B$. The ciphertext $\bar{A}$ is composed of d copies of each column of A horizontally concatenated - d copies of column 1 in the first square, d copies of column 2 in the second square, and so on. The ciphertext $\bar{B}$ consists of d copies of each row of B vertically concatenated - d copies of row 1 in the first square, d copies of row 2 in the second square, and so on.

The ciphertext $\bar{C}$ is obtained in three steps, where Figure 4 illustrates steps 1 and 2 and Figure 5 illustrates the third:

1. The ciphertexts $\bar{A}$ and $\bar{B}$ are multiplied: $\bar{C}_0 = \mathbf{Mult}(\bar{A}, \bar{B})$.
2. The squares of $\bar{C}_0$ are summed together in $\log d$ steps. This is achieved by taking $\bar{C}_0$, shifting its squares down by 1 and adding that to $\bar{C}_0$ to get $\bar{C}_1$. Then shift the squares of $\bar{C}_1$ down by 2 and add that to $\bar{C}_1$ to get $\bar{C}_2$ and so on.
3. Perform **cMult** with the resulting ciphertext from the previous step and a plaintext of 1's in the first square and zeroes elsewhere to mask out the all squares except the first. This returns $\bar{C}$, with $A \times B$ in the first square and zeroes in the rest of the slots.

The resulting ciphertext is ready for another matrix multiplication or can be decrypted and decoded to $A \times B$.

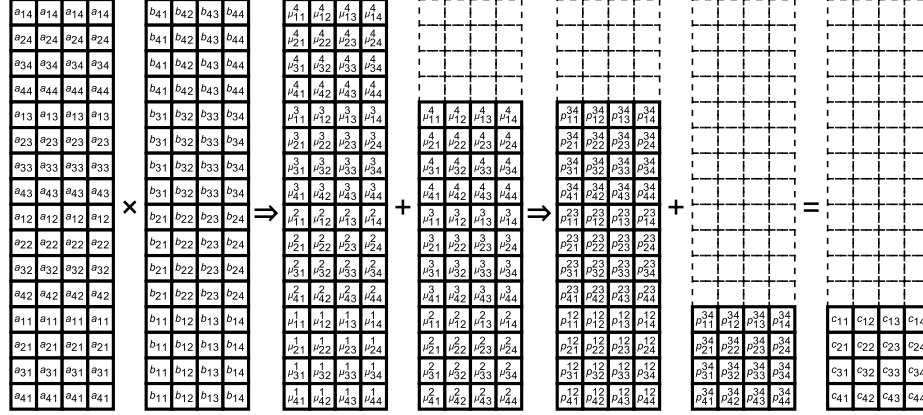


Fig. 4: Matrix multiplication algorithm starting with ciphertexts $\bar{A}$ and $\bar{B}$. The product is taken to produce a ciphertext containing the partial products, denoted $\mu_{ij}^k = a_{ik}b_{kj}$, of the matrix product. In this example, the matrix dimension is four, so the squares of this ciphertext need only be shifted $\log_2(4) = 2$ times to sum all of the partial products. These values are denoted by $p_{kj}^{ab} = \mu_{kj}^a + \mu_{kj}^b$ to represent the intermediate summing of the partial products. The resulting ciphertext contains the encrypted values of $A \times B$ in the first square, where $c_{ij} = \sum_{k=1}^4 \mu_{ij}^k = \sum_{k=1}^4 a_{ik}b_{kj}$.

The formal correctness of this algorithm is shown by considering the matrices $A = \{a_{i,j}\}_{i,j \in [d]}$, $B = \{b_{i,j}\}_{i,j \in [d]}$ with columns a_i, b_i respectively for $i \in [d]$. We denote $P_k = \{a_{i,k}b_{k,j}\}_{i,j \in [d]}$ to be the square matrix obtained from multiplying A, B from above for $k \in [d]$. Then

$$\sum_{k=1}^d P_k = \left\{ \sum_{k=1}^d a_{i,k}b_{k,j} \right\}_{i,j \in [d]} = \{a_i^T \cdot b_j\}_{i,j \in [d]} = A \times B$$

3.2 Matrix Encoding

Let d be the side length of the input square matrices and let m be a power of 2. We define an ordering on the slots of the plaintext by arranging $j \in \mathbb{Z}_m^\times$ to form a 3-dimensional tensor with side lengths $d \times d \times \frac{m}{2d^2}$. One can imagine a collection of vertically stacked $d \times d \times d$ cubes. In each cube, there are $d \times d \times d$ squares, perpendicular to the bottom of the tensor. The first square is the one facing outwards on the cube. Each cube will contain one matrix encoding, where entries from the matrix are mapped row-wise into the first square in the cube.

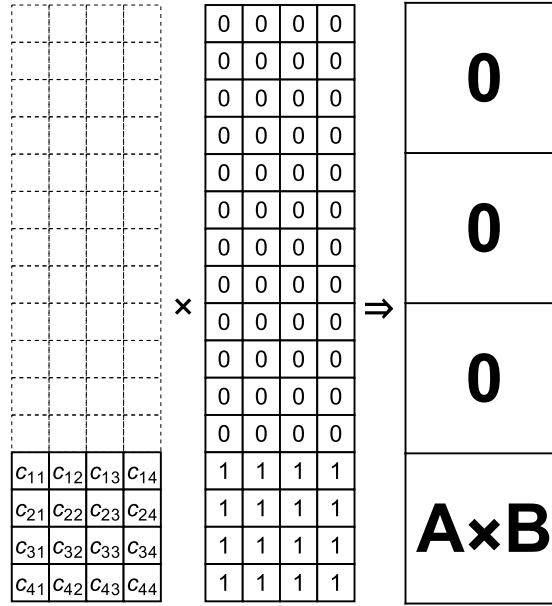


Fig. 5: The last step in the algorithm is to mask out all of the slot data outside of the first square, returning a ciphertext containing the encrypted values of $A \times B$ in the first square and zeroes everywhere else.

The ordering is uniquely determined by building the height, width, and length of the tensor from the element $1 \in \mathbb{Z}_m^\times$. Using these values, the entire tensor representation of the ciphertext can be uniquely determined. The height, which is associated with the number of squares, is determined with multiplication by an element of order d^2 . The width, which counts the number of rows in a square, is determined with multiplication by an element of order d . The length, or the number of columns, is determined by two values, one of order $m/4$ and one of order 2. The first value determines the first half of the columns and the second determines the second half of the columns. It will become apparent in the next section when we define the three-dimensional rotations why these orderings are important.

For an explicit example, we may consider $m = d^3$. The elements $3^{d/2}$ and $3^{d^2/2}$ have orders d^2 and d respectively in $\mathbb{Z}_m^\times$ which can be used to select slots to build the squares and the rows. The top left slot of each square in our ciphertext tensor representation is $1, 3^{d/2}, 3^d, 3^{3d/2}, \dots$, and the first element in each row is $1, 3^{d^2/2}, 3^{d^2}, 3^{3d^2/2}, \dots$. The remaining slots for the length are determined by 3 and -1 as described above since 3 has order $m/4$ and -1 has order 2. The first element of the first half of the columns is $1, 3, 9, 27, \dots$, and the first element of the second half of the columns is $-3, -9, -27, -81, \dots$ where the negative simply refers to the equivalent element in $\mathbb{Z}_m^\times$. Further details are provided in the Appendix explaining the order of these elements.

We will denote our matrix encoding with ξ , which takes a $d \times d$ matrix and encodes it into the first square of the first cube. Generally speaking in regards to batching, this can be extended for $k \leq n/d^3$ matrices, all encoded into the first square of the first k cubes.

In the next section, we will define row, column, and square rotations in the cipherspace dependent on this ordering. In particular, when a rotation is performed on a matrix encoded and encrypted into a ciphertext, subsequence decryption and decoding following the row, column, or square rotation operation returns the original matrix with the corresponding rotation applied on the entries in the matrix.

3.3 3-Dimensional Rotations

Our algorithm utilizes three-dimensional rotations of the ciphertext tensor representation, enabling more flexible slot movement. The name three-dimensional rotation refers to the permutation of rows, columns, and squares similar to a Rubik's cube. For visualization, center the tensor at the origin with the $d \times d$ side parallel to the xy plane. Rotations preserving a plane's slot arrangement are defined as 'swap rows,' 'swap squares,' and 'swap columns'.

Our matrix multiplication requires one cube per matrix multiplication in the ciphertext, so it suffices to reduce to talking about a single cube instead of the entire tensor. The cube can be flattened into a $d^2 \times d$ matrix of slots where the i th $d \times d$ square in the matrix is counted bottom to top, for $i \in [1, \dots, d]$.

We define a set of rotational operations on the slots in the cube, based on the previous matrix encoding. While all the rotations are motivated by 3-dimensional rotations, only one of them truly preserves the ordering of the slots after application and the other two are really *partial* 3-dimensional rotations in the sense that they only permute some of the slots as expected. In particular, the 'swap rows' operation maintains the sequential order of slots within each row with wraparound, while the 'swap squares' and 'swap columns' operations do not preserve this sequential order in wraparound. Figure 6 illustrates each of these operations with dotted lines and blank slots representing the perturbed slots. This discrepancy does not pose any limitations within the context of our proposed algorithm, however should be pointed out for completeness to the reader.

Formally, let $d = 2^k$ represent the desired matrix dimension for $k \geq 0$, and let $m \geq d^3$ be a power of 2. Consider the elements $g_1, g_2, g_3 \in \mathbb{Z}_m^\times$ such that the order of each is d, d^2 and $\frac{m}{4}$ respectively. Let c be a ciphertext and define the 3-dimensional rotations as:

- **SwapRows**(c, i) - swaps rows i row-positions up in each square, wrapping around the d th row. The j th row becomes the $j - i \pmod{d}$ th row. If $i < 0$, rows are swapped down following the same rules. Since m is a power of 2, this is explicitly defined by applying the Galois element $3^{g_1^i}$. For example, if $m = d^3$, and $i = 4$, then the resulting Galois element is $3^{(d^2/2)^4}$, reduced to an element in $\mathbb{Z}_m^\times$.

19	20	21	22	23	24	10	11	12	20		
22	23	24	25	26	27	13	14	15	23		
25	26	27	19	20	21	16	17	18	26		
10	11	12	13	14	15	1	2	3	11		
13	14	15	16	17	18	4	5	6	14		
16	17	18	10	11	12	7	8	9	17		
1	2	3	4	5	6				2		
4	5	6	7	8	9				5		
7	8	9	1	2	3				8		
Original	SwapRow	SwapSqs	SwapCol								

Fig. 6: The left-hand ciphertext has slots enumerated 1-27, starting with the top left slot in the first square, working going left to right, top to bottom, first to last square. The following ciphertexts represent shifting one row up, one square up, and one column left. The dotted, empty slots represent slots that have been perturbed after the **SwapSqs** or **SwapCols** call.

- **SwapSqs**(c, i) - swaps squares i square-positions down in the ciphertext. The first square or the front-facing square becomes the i th. Any squares wrapping around will have their slots perturbed. If $i < 0$, squares are swapped up following the same rules. Since m is a power of 2, this is explicitly defined by applying the Galois element $3^{g_2^i}$. For example, if $m = d^3$, and $i = 4$, then the resulting Galois element is $3^{(d/2)^4}$, reduced to an element in $\mathbb{Z}_m^\times$.
- **SwapCols**(c, i) - for $i \in [1, \frac{d}{2} - 1]$, swaps columns i to the left with perturbed wraparound on the $\frac{d}{2}$ th column. For $i = \frac{d}{2}$, swaps the first and second half of the columns. The perturbed wraparound means any columns wrapping around will have their slots perturbed. If $i < 0$, columns are swapped to the right following the same rules. Since m is a power of 2, $g_3 = 3$ for $i \in [1, \frac{d}{2} - 1]$ and -1 for $i = \frac{d}{2}$ regardless of the value of d . This function is explicitly defined by applying the Galois element 3^i for $i \in [1, \frac{d}{2} - 1]$ and -1 for $i = \frac{d}{2}$. So if $i = 4$, the corresponding Galois element to shift the columns four to the left is 3^4 .

The following are scalar elements used in our algorithm. The functions *MaskCols*, *MaskRows* and *MaskSqs* are italicized to denote that they can be precomputed in the plaintext space.

- *MaskCols*(i): slots in the i th column of each square are 1 and 0 otherwise.
- *MaskSqs*(i): slots in the i th square are 1 and 0 otherwise.
- *MaskRows*(i): slots in the i th row of each square are 1 and 0 otherwise.

3.4 Computing $\bar{A}, \bar{B}, \bar{C}$

To calculate $A \times B$, we first compute the intermediate ciphertexts $\bar{A}$ and $\bar{B}$ necessary to reduce the ciphertext-ciphertext multiplication operation to one and then compute $\bar{C}$ containing the encrypted $A \times B$. The ciphertext $\bar{A}$ is constructed by using iterative applications of **SwapSqs** and **SwapCols** to move the i th column of A into the first column of the i th square. Figure 7 and Figure 8 offer helpful visuals of this procedure.

The goal in computing $\bar{A}$, and in $\bar{B}$ and $\bar{C}$ as will soon be shown, is to leverage the next iterative step's call of **SwapCols** to achieve $O(\log(d))$ rotation operations instead of $O(d)$. For example, four stacked columns, or four squares such that the i th square contains the i th matrix column as its first column, can be computed in two calls instead of three. First **SwapCols** and **SwapSqs** are called with inputs of 1 each. This will shift the second column of the matrix the first column position of the second square. This ciphertext is added to the first so that the first square contains the normal ordering of the matrix, the second square's first column contains the second column of the matrix, and the rest of the squares are zero. Then **SwapCols** and **SwapSqs** are called with inputs of 2 each. This will shift the first two square up two square positions and shift each of their columns such that the third square contains the third matrix column in its first column position and the fourth square contains the fourth matrix column into its first column position.

After the columns from A can be stacked, the rest of the columns in each of the squares are masked out by **cMult**, and this iterative **SwapCol** process is repeated to "fill in" the rest of the ciphertext. The formal process is written in Algorithm 1.

Algorithm 1 Computing Intermediate Ciphertext $\bar{A}$

```

1: function  $\bar{A}_{\text{BAR}}(A)$  ▷ Where  $A$  -  $d \times d$  matrix
2:    $\bar{A}_{0,0} \leftarrow \xi(A)$  ▷  $\xi$  encodes  $A$ 
3:   for  $i = 0$  to  $\log_2(d/2)$  do
4:      $\bar{A}_{0,i+1} \leftarrow \text{Add}(\text{SwapSqs}(\text{SwapCols}(\bar{A}_{0,i}, 2^i), 2^i))$ 
5:   end for
6:    $\bar{A}_{1,0} \leftarrow \text{cMult}(\bar{A}_{0,\log_2(d)}, \text{MaskCols}(1))$ 
7:   for  $i = 0$  to  $\log_2(d/2)$  do
8:      $\bar{A}_{1,i+1} \leftarrow \text{Add}(\text{SwapCols}(\bar{A}_{1,i}, -2^i))$ 
9:   end for
10:   $\bar{A} \leftarrow \bar{A}_{1,\log_2(d)}$ 
11:  return  $\bar{A}$ 
12: end function

```

Similarly, the intermediate ciphertext for $\bar{B}$ can be computed by performing iterative applications of **SwapSqs** and **SwapRows** to move the i th row of B to the first row of the i th square of the ciphertext. Then **cMult** is used to isolate the first row in each square and **SwapRows** is called again to iteratively

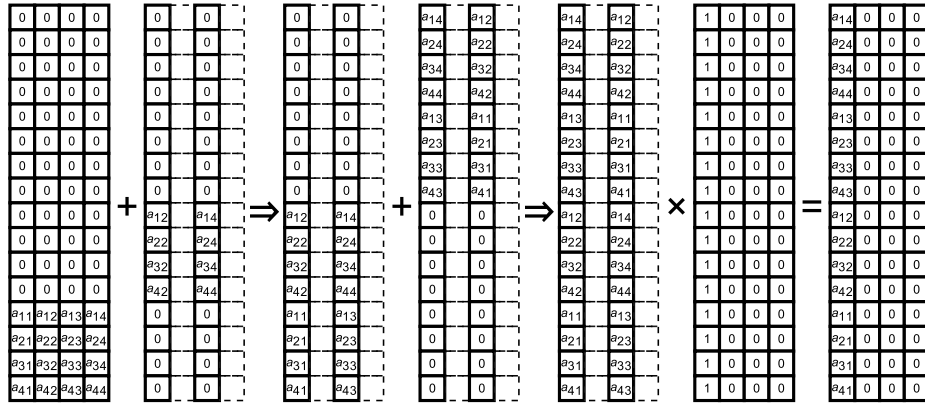


Fig. 7: Setting the first column of the i th square of the ciphertext $\bar{A}$ with the i th column of the matrix A and setting every other slot to 0. In this example, $d = 4$. The left-hand side starts with A encoded into $\bar{A}$, shifts the columns left by one, shifts the squares up by one, and sums the resulting ciphertext with the first. The dashed lines represent the slots that are perturbed by the shift and not needed by future steps. This column-square shifting process is repeated with a shift of two columns and two squares, and then **cMult** is applied to mask out rows 2-4 from each square. The total steps are $\log_2 d = 2$ **SwapCols**, **SwapSqs**, and **Add** calls.

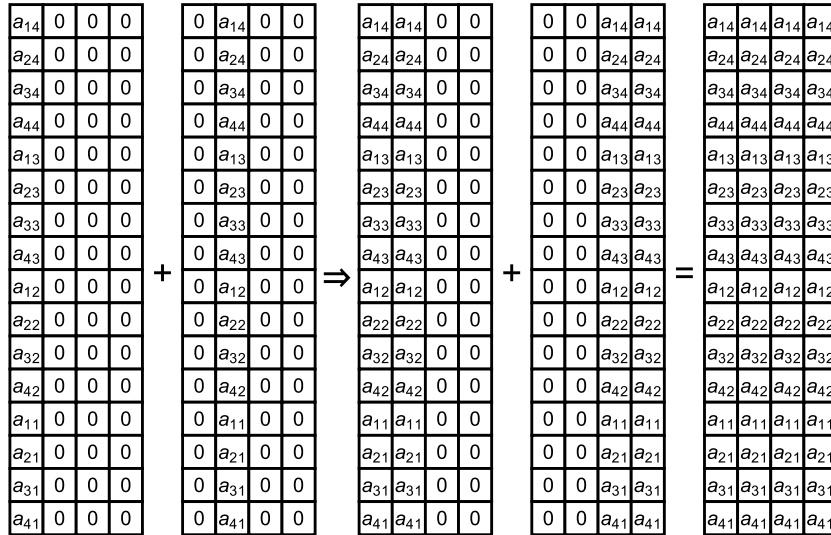


Fig. 8: Setting the i th square of $\bar{A}$ to contain four copies of the i th column of the matrix A . Starting from the left, the first column is shifted right once and added to the first ciphertext. Then the columns of the resulting ciphertext are shifted right twice. The i th square in the final ciphertext contains four copies of the i th column of A . The total steps are $\log_2 d = 2$ **SwapCols** and **Add** calls.

fill out each square with copies of its respective first row. Pictorially, this is shown in Figure 9 and Figure 10. This algorithm is similar to the construction of $\bar{A}$ except instead of relocating the columns of each square, masking out the unwanted columns, and filling in the square with copies of the respective column, $\bar{B}$ is built by rearranging and isolating the rows of the matrix. The algorithm is given in Algorithm 2.

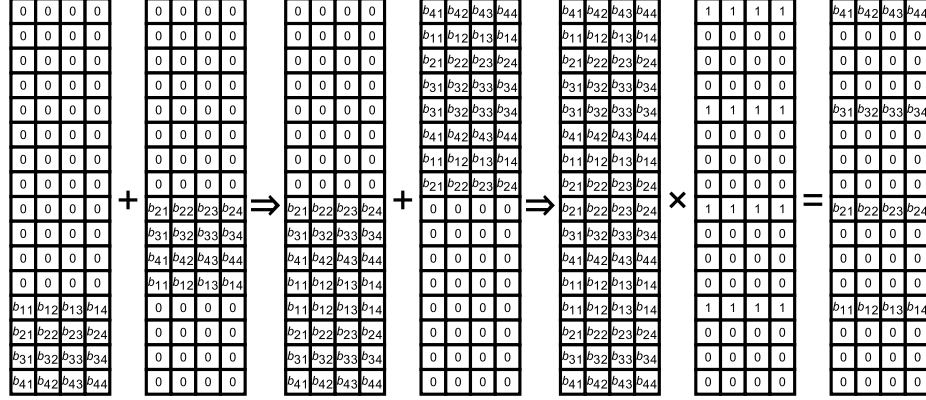


Fig. 9: Setting the first row of the i th square of the ciphertext $\bar{B}$ with the i th row of the matrix B and setting every other slot to 0. In this example, $d = 4$. The left-hand side starts with B encoded into $\bar{B}$, shifts the rows up by one, shifts the squares up by one, and sums the resulting ciphertext with the first. This process is repeated with a shift of two rows and two squares, and then **cMult** is applied to mask out rows 2-4 from each square. The total steps are $\log_2 d = 2$ **SwapRows**, **SwapSqs**, and **Add** calls.

Following the calculations of $\bar{A}$ and $\bar{B}$, the final stage of the encrypted matrix multiplication is to compute the Hadamard product of $\bar{A}$ and $\bar{B}$ with **Mult** and to rearrange the resulting ciphertext to contain the encrypted product $A \times B$ in the appropriate slots. This is the algorithm given in the overview of the encrypted matrix multiplication algorithm as it contains the core ciphertext-ciphertext multiplication operation call. The final ciphertext containing the encrypted product is called $\bar{C}$. The algorithm for $\bar{C}$ intakes $\bar{A}$ and $\bar{B}$ and is divided into three steps:

1. The ciphertexts $\bar{A}$ and $\bar{B}$ are multiplied: $\bar{C}_0 = \mathbf{Mult}(\bar{A}, \bar{B})$.
2. The squares of $\bar{C}_0$ are summed together.
3. The resulting squares with the exception of the first are masked out, leaving $A \times B$ in the first square.

Unlike the previous two algorithms for $\bar{A}$ and $\bar{B}$, this only requires one for-loop as $\bar{C}$ only needs to sum the partial products resulting from **Mult**. The iterative call of **SwapSqs** uses the same techniques as calls for **SwapCols** and **SwapRows** in $\bar{A}$ and $\bar{B}$ respectively to achieve the $O(\log d)$ number of rotation

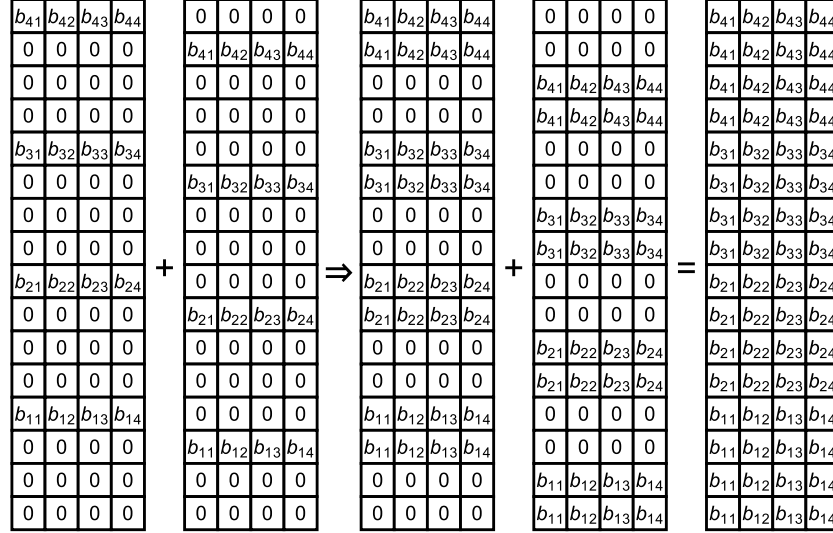


Fig. 10: Setting the i th square of $\bar{B}$ to contain four copies of the i th row of the matrix B . Starting from the left, the first row is shifted down once and added to the first ciphertext. Then the rows of the resulting ciphertext are shifted down twice. The i th square in the final ciphertext contains four copies of the i th row of B . The total steps are $\log_2 d = 2$ **SwapRows**, **SwapSqs**, and **Add** calls.

Algorithm 2 Computing Intermediate Ciphertext $\bar{B}$

```

1: function  $B\_BAR(B)$  ▷ Where  $B$  -  $d \times d$  matrix
2:    $\bar{B}_{0,0} \leftarrow \xi(B)$  ▷  $\xi$  encodes  $B$ 
3:   for  $i = 0$  to  $\log_2(d/2)$  do
4:      $\bar{B}_{0,i+1} \leftarrow \text{Add}(\text{SwapSqs}(\text{SwapRows}(\bar{B}_{0,i}, 2^i)))$ 
5:   end for
6:    $\bar{B}_{1,0} \leftarrow \text{cMult}(\bar{B}_{0,\log_2(d)}, \text{MaskRows}(1))$ 
7:   for  $i = 0$  to  $\log_2(d/2)$  do
8:      $\bar{B}_{(1,i+1)} \leftarrow \text{Add}(\text{SwapSqs}(\bar{B}_{(1,i)}, -2^i))$ 
9:   end for
10:   $\bar{B} \leftarrow \bar{B}_{1,\log_2(d)}$ 
11:  return  $\bar{B}$ 
12: end function

```

operations. The last call of **cMult** is necessary to clear away the additional data from the other slots to ready the ciphertext for potential further applications of the algorithm. The process to compute $\bar{C}$ is formally detailed in Algorithm 3.

Algorithm 3 Computing Ciphertext $\bar{C}$

```

1: function C_BAR( $\bar{A}, \bar{B}$ )                                     ▷ Where  $\bar{A}, \bar{B}$  ciphertexts
2:    $\bar{C}_0 \leftarrow \mathbf{Mult}(\bar{A}, \bar{B})$ 
3:   for  $i = 0$  to  $\log_2(d/2)$  do
4:      $\bar{C}_{i+1} \leftarrow \mathbf{Add}(\mathbf{SwapSqs}(\bar{C}_i, -2^i))$ 
5:   end for
6:    $\bar{C} \leftarrow \mathbf{cMult}(\bar{C}_{\log_2 d}, \mathbf{MaskSqs}(1))$ 
7:   return  $\bar{C}$ 
8: end function

```

The full algorithm is simply encoding the matrices A and B into $\bar{A}$ and $\bar{B}$ respectively and calculating $\bar{C}$, as shown in Algorithm 4. From there, the resulting ciphertext can be decrypted and decoded to access the matrix product $A \times B$ or can be used for further encrypted matrix multiplications within the encrypted space.

Algorithm 4 Encrypted Matrix Multiplication for $A \times B$

```

function OURMATRIXMULTIPLICATION( $A, B$ )
   $\bar{A} \leftarrow \mathbf{A\_bar}(A)$                                      ▷ Intermediate ciphertext for  $A$ 
   $\bar{B} \leftarrow \mathbf{B\_bar}(B)$                                      ▷ Intermediate ciphertext for  $B$ 
   $\bar{C} \leftarrow \mathbf{C\_bar}(\bar{A}, \bar{B})$  return  $\bar{C}$ 
end function

```

3.5 Operation Analysis

Given $d \times d$ matrices, our encrypted matrix multiplication algorithm consumes one **Mult**, three **cMult**, $O(\log d)$ **Rot** and $O(\log d)$ **Add** per execution, has a multiplicative depth of $1\mathbf{Mult} + 2\mathbf{cMult}$, and requires a minimum ciphertext size of d^3 slots.

The number of operations for homomorphic addition, rotation, plaintext-ciphertext multiplication and ciphertext-ciphertext multiplication are given in Table 2.

The calculations for $\bar{A}$ and $\bar{B}$ both include two for loops of size $\log d - 1$ iterating over two rotation and one addition applications, see lines 3-5 and lines 7-9 in Algorithm 1 and lines 3-5 and lines 7-9 in Algorithm 2. Both algorithms consume one plaintext-ciphertext multiplication on line 6. The calculation of $\bar{C}$ uses one ciphertext-ciphertext product, 1 plaintext-ciphertext multiplication,

Table 2: Number of Operations and Multiplicative Depth for Each Stage

	# of Add	# of Rot	# of cMult	# of Mult	Depth
A	$2(\log_2(d) - 1)$	$3(\log_2(d) - 1)$	1	-	1 cMult
B	$2(\log_2(d) - 1)$	$3(\log_2(d) - 1)$	1	-	1 cMult
C	$\log_2(d) - 1$	$\log_2(d) - 1$	1	1	1 Mult + 1 cMult
Total	$5(\log_2(d) - 1)$	$7(\log_2(d) - 1)$	3	1	1 Mult + 2 cMult

and $\log_2 d - 1$ rotations and additions, see lines 2, 6, and 3-5 respectively in Algorithm 3). The operations for *MaskCols*, *MaskSqr*s, *MaskRows* are not included because they can be precomputed as plaintexts outside of the algorithm execution and are independent of the data in the input matrices.

3.6 Advanced Extensions of Encrypted Square Matrix Multiplication Algorithm

Matrix Packing Given a $d \times d$ matrix and a ciphertext space with d^3 dividing n , $n = cd^3$ for $c \geq 1$, we can extend our algorithm to consider multiple multiplications occurring in parallel with no additional computational cost or noise. Our algorithm operates over all slots in the ciphertext, so if $d^3 \ll n$, many slots will be unutilized. For example, if $n = 2^{15}$ and $d = 16$, a single 16×16 matrix multiplication will use 2^{12} slots, which is only 12.5% of the slots in the ciphertext. For $d = 8$, this drops even lower to about 1.6% slot usage. To maximize efficiency, it is preferred to pack in additional matrices, such that a ciphertext can contain at most $\sigma = n/d^3$ matrices. So in the example of $n = 2^{15}$ and $d = 16$, this would allow for eight additional multiplications that can occur in parallel, and for $n = 2^{15}$ and $d = 8$, this would allow for 64 additional multiplications.

Let $k \leq c$ and let $\mathcal{S}_A = \{A_1, A_2, \dots, A_k\}$ and $\mathcal{S}_B = \{B_1, B_2, \dots, B_k\}$. We define the internal function *LoadMatrices* which takes in an ordered set of k matrices and returns a ciphertext loaded with the k matrices. The function *LoadMatrices* encodes the k matrices using ξ and then performs **SwapSrqs** to shift the i th matrix ciphertext into the first square of i th cube in the ciphertext. Then $\bar{A}$ and $\bar{B}$ can be computed as previously outlined in Algorithm 1 and Algorithm 2. The execution of the encrypted matrix multiplication will produce the products $A_1 \times B_1, A_2 \times B_2, \dots, A_k \times B_k$ in the first square of the first k cubes in the resulting ciphertext. Using a reverse function of *LoadMatrices* called *UnloadMatrices*, the individual products can be recovered.

Rectangular Matrix Multiplication Our encrypted matrix multiplication can be modified to accommodate rectangular matrices, either of the form $l \times d$ and $d \times d$, $d \times d$ and $d \times k$, or more generally, $l \times d$ and $d \times k$, where l, k divide d . The encoding function ξ will map the first column of the matrix entries to the first column in the first ciphertext square or the last row of the matrix entries to the last row in the first ciphertext square, depending on the size of the matrix. Line 7 in Algorithm 1 and Algorithm 2 can be modified to go up to l or k to

reduce the number of rotation and addition operations, called RecA_bar and RecB_bar respectively. Then to calculate $\bar{C}$, we apply Algorithm 3 without any additional changes.

Algorithm 5 Encrypted Rectangular Matrix Multiplication for $A \times B$

function OURRECTANGULARMATRIXMULTIPLICATION(A, B) $\triangleright A$ is $l \times d$ matrix,
 B is $d \times k$ matrix, $l, k | d$
 $\bar{A} \leftarrow \text{RecA_bar}(A)$ $\triangleright$ Intermediate ciphertext for A
 $\bar{B} \leftarrow \text{RecB_bar}(B)$ $\triangleright$ Intermediate ciphertext for B
 $\bar{C} \leftarrow \text{C_bar}(\bar{A}, \bar{B})$ **return** $\bar{C}$
end function

4 Implementation

This section provides a comparative analysis of our work in terms of runtime and noise generation against the most recent comparable encrypted matrix multiplication algorithm [28]. Both algorithms are implemented in Microsoft SEAL in the BFV scheme and use ciphertexts containing 32^3 slots. Data was collected with a Windows 11 machine using a Intel64 Family 6 Model 170 Stepping 4 GenuineIntel 2300 Mhz Processor. Experiments were run for square matrices of dimension 2, 4, 8, and 16 with 50 trials for each dimension. Notably, the algorithm proposed by Rizomiliotis et al. is limited to matrices of dimension 16, while our algorithm was further tested with size 32 matrices. The results are summarized in Table 3 and Table 4 and grouped by matrix dimension.

Table 3: Average performance comparison of our encrypted matrix multiplication algorithm with Rizomiliotis et al. [28] over 50 trials for 2, 4, 8, and 16 square matrices by runtime for each ciphertext relating to the matrices $A, B, C = A \times B$, as well as the total algorithm runtime and noise. Runtime is denoted ‘RT’ and matrix dimension is denoted ‘Dim.’.

Method	Dim.	RT A (s)	RT B (s)	RT C (s)	Total RT (s)
[28]	2	15.37	11.03	16.27	42.68
This work	2	14.53	14.47	16.74	45.74
[28]	4	49.40	40.17	21.07	110.64
This work	4	28.70	28.59	21.54	78.84
[28]	8	128.74	109.76	32.29	270.80
This work	8	45.64	45.54	28.06	119.25
[28]	16	336.64	304.90	42.14	683.68
This work	16	66.88	67.56	36.61	171.04
This work	32	82.66	82.52	41.56	206.74

Table 4: Average performance comparison of our encrypted matrix multiplication algorithm with Rizomiliotis et al. [28] over 50 trials for 2, 4, 8, and 16 square matrices by noise for each ciphertext relating to the matrices $A, B, C = A \times B$, as well as the total algorithm runtime and noise. Matrix dimension is denoted ‘Dim.’.

Method	Dim.	Noise A (bits)	Noise B (bits)	Noise C (bits)	Total Noise (bits)
[28]	2	19.14	6.98	33.00	59.12
This work	2	23.10	6.62	56.60	86.32
[28]	4	21.12	6.00	33.16	60.28
This work	4	23.86	8.02	56.44	88.32
[28]	8	22.08	6.22	33.86	62.16
This work	8	26.10	7.18	56.86	90.14
[28]	16	24.17	6.04	33.92	64.12
This work	16	28.25	6.62	56.12	91.00
This work	32	29.40	7.12	55.82	92.34

Our work shows a significant reduction in total runtime across matrix dimension compared to the cited work, as documented in Figure 11. In addition to having lower runtime, our algorithm scales logarithmically by matrix dimension versus a linear increase. Our trials showed a relative improvement factor per dimension of $0.93\times$, $1.40\times$, $2.27\times$, and $4.00\times$ for dimensions 2, 4, 8, and 16 respectively. This appears consistent with the predicted logarithmic complexity, indicates better scalability for large dimension matrices.

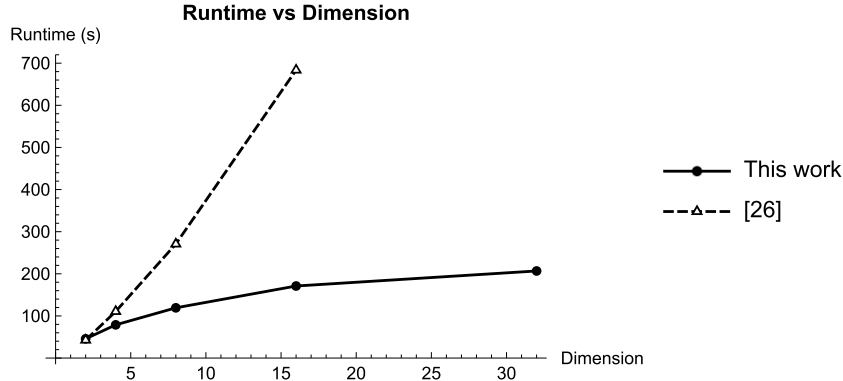


Fig. 11: Comparison of runtime in seconds vs Rizomiliotis and Triakosia [28] over all possible matrix sizes in Microsoft SEAL implementation.

The difference in noise generation between our work and Rizomiliotis and Triakosia’s remains relatively constant with an average difference of 27.20, 28.04, 27.98, and 26.88 bits for dimensions 2, 4, 8, and 16 respectively, as seen in Figure

12. This matches the theoretic multiplicative depth analysis of a fixed increase in noise. For a concrete example from the data in Table 3, the budget for 32^3 slots is 800 bits, so our algorithm can be executed for $800/91 \approx 8.8$ runs vs $800/64 \approx 12.5$ runs with a performance gain of 512.6s before bootstrapping. This makes our algorithm advantageous in situations where there is a smaller number of operations needed to be performed with a tighter timing constraint.

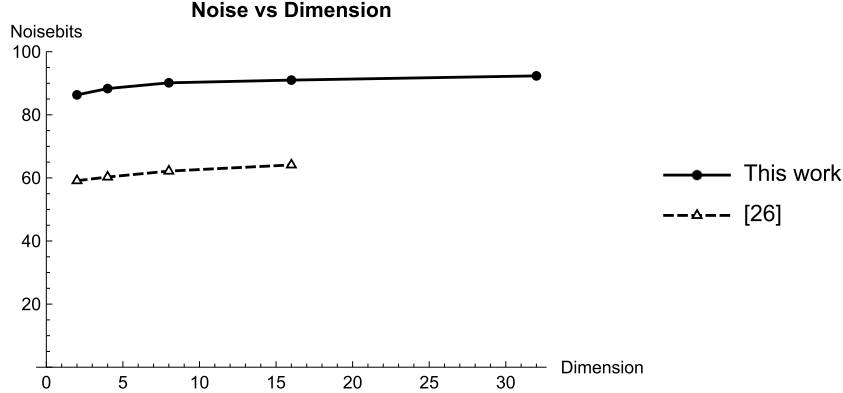


Fig. 12: Comparison of noise in bits vs Rizomiliotis and Triakosia [28] over all possible matrix sizes in Microsoft SEAL implementation.

5 Conclusion

In conclusion, our work has demonstrated a significant improvement in complexity of necessary homomorphic operations for encrypted matrix multiplication, with notable reductions in plaintext-ciphertext multiplication operations from $O(d)$ to $O(1)$ and rotation operations from $O(d)$ to $O(\log d)$ for square matrices of dimension d . While our algorithm has greater plaintext-ciphertext multiplicative depth than other recent methods [28], it is our belief the substantial gains in speed outweigh the constant increase in noise consumption.

Fully homomorphic encryption remains a relatively new field of research, especially in the applied setting, and as such, there are several directions our work can be extended. First, the algorithm can naturally be generalized to encrypted tensor products, as the “swap” operations we defined in Section 3 mimic the behavior of rotations of elements in a three-dimensional tensor. Such an extension would have immediate application in secure machine learning frameworks, which heavily utilize tensor operations. Second, matrix diagonalization seems likely as a viable method of reducing ciphertext size requirements.

6 Acknowledgements

We would like to sincerely thank everyone involved who helped us bring this work together into a final paper. Especially, thank you to Anton Leykin for insightful discussions and keen attention to detail, thank you to Greg Mohler for reviewing multiple drafts, and thank you to Sasha Boldyreva for familiarity with the cryptography area. Thank you very much to Lee Lerner and Will Stuckey for their ongoing support throughout this work from start to finish. This paper marks the enjoyable beginning to our journey in the field of FHE.

References

1. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(4), 114–148 (Aug 2021). <https://doi.org/10.46586/tches.v2021.i4.114-148>, <https://tches.iacr.org/index.php/TCHES/article/view/9062>
2. Helib (2024), <https://github.com/homenc/HELlib>
3. Openfhe (2024), <https://github.com/openfheorg>
4. Badawi, A.A., Alexandru, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Pascoe, C., Polyakov, Y., Quah, I., R.V., S., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915 (2022), <https://eprint.iacr.org/2022/915>
5. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: Fully homomorphic encryption without bootstrapping. Cryptology ePrint Archive, Paper 2011/277 (2011), <https://eprint.iacr.org/2011/277>
6. Chen, H., Han, K., Huang, Z., Jalali, A., Laine, K.: Simple encrypted arithmetic library (2017), <https://www.microsoft.com/en-us/research/wp-content/uploads/2017/12/sealmanual.pdf>
7. Chen, J., Yang, L., Wu, W., Liu, Y., Feng, Y.: Homomorphic matrix operations under bicyclic encoding. Cryptology ePrint Archive, Paper 2024/1762 (2024), <https://eprint.iacr.org/2024/1762>
8. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) Advances in Cryptology – ASIACRYPT 2017. pp. 409–437. Springer International Publishing, Cham (2017)
9. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption library (August 2016), <https://tfhe.github.io/tfhe/>
10. Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. Cryptology ePrint Archive, Paper 2021/729 (2021), <https://eprint.iacr.org/2021/729>
11. Gao, J., Gao, Y.: Gms: an efficient fully homomorphic encryption scheme for secure outsourced matrix multiplication. The Journal of Supercomputing **80**(18), 26435–26461 (Dec 2024). <https://doi.org/10.1007/s11227-024-06449-3>
12. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing. p. 169–178. STOC ’09, Association for Computing Machinery, New York, NY, USA (2009). <https://doi.org/10.1145/1536414.1536440>

13. Halevi, S., Shoup, V.: Algorithms in helib. In: Garay, J.A., Gennaro, R. (eds.) *Advances in Cryptology – CRYPTO 2014*. pp. 554–571. Springer Berlin Heidelberg, Berlin, Heidelberg (2014)
14. Halevi, S., Shoup, V.: Design and implementation of helib: a homomorphic encryption library. *Cryptology ePrint Archive*, Paper 2020/1481 (2020), <https://eprint.iacr.org/2020/1481>, <https://eprint.iacr.org/2020/1481>
15. Jacobs, D.B.: Data protection in virtual environments (dprive), <https://www.darpa.mil/program/data-protection-in-virtual-environments>
16. Jiang, X., Kim, M., Lauter, K., Song, Y.: Secure outsourced matrix computation and application to neural networks. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. p. 1209–1222. CCS '18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3243734.3243837>, <https://doi.org/10.1145/3243734.3243837>
17. Jiang, X., Kim, M., Lauter, K., Song, Y.: Secure outsourced matrix computation and application to neural networks. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. p. 1209–1222. CCS '18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3243734.3243837>, <https://doi.org/10.1145/3243734.3243837>
18. Kosieradzki, S., Qiu, Y., Kogiso, K., Ueda, J.: Rewrite rules for automated depth reduction of encrypted control expressions with somewhat homomorphic encryption. In: *2022 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*. pp. 804–809 (2022). <https://doi.org/10.1109/AIM52237.2022.9863395>
19. Kosieradzki, S., Zhao, X., Kawase, H., Qiu, Y., Kogiso, K., Ueda, J.: Secure teleoperation control using somewhat homomorphic encryption. *IFAC-PapersOnLine* **55**(37), 593–600 (2022). <https://doi.org/10.1016/j.ifacol.2022.11.247>, <https://www.sciencedirect.com/science/article/pii/S2405896322028907>, 2nd Modeling, Estimation and Control Conference MECC 2022
20. Kwon, H.B., Kosieradzki, S., Blevins, J., Ueda, J.: Encrypted coordinate transformation via parallelized somewhat homomorphic encryption for robotic teleoperation. In: *2023 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*. pp. 228–233 (2023). <https://doi.org/10.1109/AIM46323.2023.10196122>
21. Lee, J.W., Kang, H., Lee, Y., Choi, W., Eom, J., Deryabin, M., Lee, E., Lee, J., Yoo, D., Kim, Y.S., No, J.S.: Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access* **10**, 30039–30054 (2022). <https://doi.org/10.1109/ACCESS.2022.3159694>
22. Lee, Y., Micciancio, D., Kim, A., Choi, R., Deryabin, M., Eom, J., Yoo, D.: Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. *Cryptology ePrint Archive*, Paper 2022/198 (2022). https://doi.org/10.1007/978-3-031-30620-4_8, <https://eprint.iacr.org/2022/198>
23. Jie Lu, W., Kawasaki, S., Sakuma, J.: Using fully homomorphic encryption for statistical analysis of categorical, ordinal and numerical data. *Cryptology ePrint Archive*, Paper 2016/1163 (2016). <https://doi.org/http://dx.doi.org/10.14722/ndss.2017.23119>, <https://eprint.iacr.org/2016/1163>

24. Meftah, S., Tan, B.H.M., Mun, C.F., Aung, K.M.M., Veeravalli, B., Chandrasekhar, V.: Doren: Toward efficient deep convolutional neural networks with fully homomorphic encryption. *IEEE Transactions on Information Forensics and Security* **16**, 3740–3752 (2021). <https://doi.org/10.1109/TIFS.2021.3090959>
25. Microsoft: Microsoft seal (2024), <https://github.com/microsoft/SEAL>
26. Min, Z., Yang, G., Sangaiah, A.K., Bai, S., Liu, G.: A privacy protection-oriented parallel fully homomorphic encryption algorithm in cyber physical systems. *EURASIP Journal on Wireless Communications and Networking* **2019**(1), 15 (Jan 2019). <https://doi.org/10.1186/s13638-018-1317-9>, <https://doi.org/10.1186/s13638-018-1317-9>
27. Polyakov, Y.: Practical lattice-based cryptography in palisade (2020), <https://simons.berkeley.edu/sites/default/files/docs/15667/practicallattice-basedcryptographyinpalisade.pdf>
28. Rizomiliotis, P., Triakosia, A.: (Nov 2022), <https://dl.acm.org/doi/pdf/10.1145/3560810.3564267>
29. Wang, S., and, H.H.: Secure outsourced computation of multiple matrix multiplication based on fully homomorphic encryption. *KSII Transactions on Internet and Information Systems* **13**(11), 5616–5630 (November 2019). <https://doi.org/10.3837/tis.2019.11.019>
30. Yan, X., Zhou, G., Huang, Y., Meng, W., Nguyen, A.T., Huang, H.: Secure estimation using partially homomorphic encryption for unmanned aerial systems in the presence of eavesdroppers. *IEEE Transactions on Intelligent Vehicles* pp. 1–10 (2024). <https://doi.org/10.1109/TIV.2024.3378288>
31. Zheng, X., Li, H., Wang, D.: A new framework for fast homomorphic matrix multiplication. *Cryptology ePrint Archive*, Paper 2023/1649 (2023), <https://eprint.iacr.org/2023/1649>

A Underlying Mathematics

This section is intended to provide an interested reader with more mathematical details on why we are able to represent elements in the plaintext and ciphertext space as 3-dimensional tensors and how we identify appropriate Galois elements to mimic 3-dimensional rotations.

We provide a more detailed overview of cyclotomic polynomials and primitive roots of unity in the setting of finite fields. Then we explain why we are able to represent ciphertexts by a collection of slots, why these slots can rotate, and how to do so. We provide a simple example explicitly showing what it means for data to be stored inside ciphertext slots and how the slots are permuted by various automorphisms.

Next we explain how we identified how to define the slot ordering for the matrix encoding such that we could define rotations to mimic 3-dimensional rotations of the slots. This is not necessary to the implementation of the algorithm, and is intended to provide more insight on how we built our encoding using a simpler isomorphism.

Lastly, we explain how to define the slot ordering more easily using an equivalent additive group. In the context of our algorithm for 2-dimensional matrices, we write a method to reduce taking powers of three, however it may be of interest for extensions in k -dimensions to consider working with an easier group. We give

an isomorphism between Galois elements and elements from an additive group and use the latter to arrange the slots as desired. We end the section with an explicit example for $d = 4$ using this isomorphism and identify the corresponding matrix encoding and 3-dimensional rotations.

A.1 Explicit Overview

We will mathematically define the plaintext space, its elements, the isomorphic “slot” representation, the rotation or automorphism operation, and the encoding of a matrix into a plaintext. We will also provide a simple example illustrating this entire process.

Setting the Stage

Definition 1 (Cyclotomic Polynomial, Primitive Roots of Unity). *Let K be a field. The m th cyclotomic, denoted $\Phi_m(x)$, refers to the unique, irreducible polynomial dividing $x^m - 1$ and no $x^k - 1$ for $k \in [1, m - 1]$. The roots of $\Phi_m(x) \in K[x]$ for $x \in K$ are referred to as the primitive m th roots of unity. If $K = \mathbb{C}$, then the primitive m th roots are $e^{2i\pi k/m}$ where $\gcd(k, m) = 1, k \in [1, m]$.*

Let $\Phi_m(X) \in \mathbb{Z}[X]$ be the m th cyclotomic polynomial with primitive m th root $\omega = e^{2i\pi/m} \in \mathbb{C}$. Given polynomials $f(X), g(X) \in \mathbb{Z}[X]$ where $f(X) \equiv g(X) \pmod{\Phi_m(X)}$, it follows since $\Phi_m(X)$ divides $f(X) - g(X)$ that $f(\omega) = g(\omega)$.

Let $\mathcal{A}$ denote the field $\mathbb{Z}[X]/(\Phi_m(X))$. If $a = [f(X) \pmod{\Phi_m(X)}] \in \mathcal{A}$, then $a(\omega) = f(\omega)$. Define the canonical form of a to be

$$\text{canon}(a) = (a(\omega^j))_{j \in \mathbb{Z}_m^\times}$$

where a is evaluated at all the primitive m th roots of unity from $\mathbb{C}$. It is well-known that these are generated by ω^α for $\alpha \in \mathbb{Z}_m^\times$.

Plaintext Representations We will give a unique representation of plaintexts in $\mathcal{A}$. Let $x := [X \pmod{\Phi_m(X)}]$ be the image of the indeterminate X . For any $f(X) \in \mathbb{Z}[X]$, there exists a unique remainder polynomial $r(x)$ such that the degree of $r(X) < \Phi_m(X)$ and $f(X) \equiv r(X) \pmod{\Phi_m(X)}$. Then $f(x) = r(x)$, so we may write $\mathcal{A} = \mathbb{Z}[x]$ with uniquely represented elements.

Galois Automorphism Let $\theta_\alpha : \mathcal{A} \rightarrow \mathcal{A}$ be the Galois automorphism mapping $f(x)$ to $f(x^\alpha)$ for $\alpha \in \mathbb{Z}_m^\times$. This is well-defined. If we take $f(x) = g(x)$, then $\Phi_m(X) \mid f(X) - g(X)$, which implies that $\Phi_m(X^\alpha) \mid f(X^\alpha) - g(X^\alpha)$. Also $\Phi_m(X) \mid \Phi_m(X^\alpha)$ since ω is a root of $\Phi_m(X^\alpha)$. Then it follows that $\Phi_m(X) \mid f(X^\alpha) - g(X^\alpha)$ and $f(x^\alpha) = g(x^\alpha)$.

Here are several more properties:

- Commutativity of composition:

$$\theta_\alpha \circ \theta_\beta = \theta_{\alpha\beta} = \theta_\beta \alpha = \theta_\beta \circ \theta_\alpha$$

$$\text{due to } \theta_\alpha \circ \theta_\beta(f(x)) = f(x^{\beta\alpha}) = f(x^{\alpha\beta}) = \theta_\beta \circ \theta_\alpha(f(x)).$$

- Bijection: there exists an inverse $\theta_{\alpha^{-1}}$

$$\theta_{\alpha} \circ \theta_{\alpha^{-1}} = \theta_1 = \theta_{\alpha^{-1}} \circ \theta_j$$

due to $\mathbb{Z}_m^{\times}$ is a group under multiplication.

Note that $\text{canon}(\theta_{\alpha}(f))$ is just a permutation of entries in $\text{canon}(f)$.

The rotation operation or Galois automorphism is determined by selecting an element from the Galois group of the field extension. In each implementation, an ordering of the canonical form of the plaintext is fixed and each rotation is defined by selecting an appropriate Galois element to create the desired effect on the slots. We give an example for a $2 \times n/2$ matrix representation of the ciphertext. Suppose m is a power of 2 and let the isomorphic form of the plaintext $f(x)$ be:

$$\begin{pmatrix} f(\eta), f(\eta^3), f(\eta^9), \dots, f(\eta^{3^{m/4-1}}) \\ f(\eta^{-1}), f(\eta^{-3}), f(\eta^{-9}), \dots, f(\eta^{-1 \cdot 3^{m/4-1}}) \end{pmatrix}$$

where the first row is evaluated at over powers of 3 of ζ_m and the second row is evaluated over powers of 3 of ζ_m^{-1} . To create a left rotation of one slot, $\mathbf{Rot}(c_1, 1)$, apply the Galois element $x \mapsto x^3$, so the plaintext $f(x^3)$ is:

$$\begin{pmatrix} f(\eta^3), f(\eta^9), f(\eta^{27}), \dots, f(\eta) \\ f(\eta^{-3}), f(\eta^{-9}), f(\eta^{-27}), \dots, f(\eta^{-1}) \end{pmatrix}$$

Isomorphism Define $\mathcal{A}_p = \mathbb{Z}_p[x]$ and choose p to be a prime such that $p \equiv 1 \pmod{m}$. In other words, p has order 1 in $\mathbb{Z}_m^{\times}$. Then $\Phi(X) \in \mathbb{Z}_p[X]$ factors as linearly into $F_1(X), \dots, F_n(X)$ where $n = \phi(m) = m/2$ and $\deg(F_i(X)) = 1$.

By the Chinese Remainder Theorem, we have a $\mathbb{Z}_p$ -algebra isomorphism [14]

$$\begin{aligned} \mathcal{A}_p &\rightarrow \mathbb{Z}_p[X]/(F_1(X)) \times \dots \times \mathbb{Z}_p[X]/(F_n(X)) \\ f(X) &\mapsto ([f(X) \pmod{F_1(X)}], \dots, [f(X) \pmod{F_n(X)}]) \end{aligned}$$

Each $F_i(X) = X - \eta_i$ for $\eta_i \in \mathbb{Z}_p$ a primitive m th root of unity. The collection η_i for $i \in [1, n]$ are all of the primitive m th roots of unity. It is well-known that given a primitive m th root of unity $\eta \in \mathbb{Z}_p$, all of the primitive m th roots of unity in $\mathbb{Z}_p$ can be generated via $j \in \mathbb{Z}_m^{\times}$.

Simple Example We give an example of a matrix encoding into a plaintext.

Choose $m = 8$, $p = 17$ and a matrix $M = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.

Then $\Phi_8(X) = X^4 + 1$ and the roots of $\Phi_8(X) \in \mathbb{Z}_{17}[X]$ are 2, 8, 9, 15. This can be calculated by testing elements in $\mathbb{Z}_{17}$ until one of order 8 is found and then generating the rest by raising that number to 3, 5, 7. So for this case, $2^8 = 1$ and $2^3 = 8$, $2^5 = 15$, $2^7 = 9$.

Next we use Lagrange interpolation to construct a plaintext $f(x) \in \mathcal{A}_p$ such that f evaluated at 2, 8, 9, 15 are entries in the matrix. We choose the encoding based on mapping increasing values of 2^j to row-wise entries of M :

- $2 \mapsto 1$
- $8 = 2^3 \mapsto 2$
- $9 = 2^7 \mapsto 3$
- $15 = 2^5 \mapsto 4$

Then $f(x) = -5x^3 - 2x - 6 = 12x^3 + 15x + 11 \in \mathcal{A}_p$ and $f(x) \mapsto (f(2), f(8), f(9), f(15)) = (1, 2, 3, 4)$.

The Galois automorphisms or possible rotations applied to f are:

- Identity: $\theta_1(f(x)) = (1, 2, 3, 4)$
- Swap Columns: $\theta_3(f(x^3)) = (2, 1, 4, 3)$
- Swap Rows: $\theta_5(f(x^7)) = (3, 4, 1, 2)$
- Swap Columns, Swap Rows: $\theta_7(f(x^5)) = (4, 3, 2, 1)$

Plaintext and Ciphertext Space Cube Behavior An equivalent statement to identifying Galois elements to generate rotations is to identify which subgroups of $\mathbb{Z}_m^\times$ act on the set of primitive m th roots of unity to return orbits of the desirable order. These subgroups in the case of row rotation and square rotation are going to each be generated by elements of order d and d^2 respectively. The column rotation is then selected based on elements that have order $m/4$, which effectively shift elements left or right or generate orbits of size $m/4$.

Identifying these elements from the multiplicative group $\mathbb{Z}_m^\times$ requires a computer to reduce elements raised to high power. During the construction of our work, we used the following well-known theorem to find an isomorphic group under addition to avoid dealing with elements of the multiplicative group.

Theorem 1 (Structure Theorem). *The roots of $\Phi_m(x)$, the primitive m th roots of unity, form a group Ω_m under exponentiation. Then*

$$\Omega_m \cong \mathbb{Z}_m^\times \cong \mathbb{Z}_2 \times \mathbb{Z}_{m/4}$$

In particular, $\psi : \mathbb{Z}_2 \times \mathbb{Z}_{m/4} \rightarrow \mathbb{Z}_m^\times$, mapping $(0, 1) \mapsto 3$ and $(1, 0) \mapsto -1$, is an isomorphism.

To simplify the identification of orbits and rotations, we can utilize this more convenient group structure. Using the standard orientation of the x, y, z axes, letting $m = 2d^3$ and the isomorphism ψ from Theorem 1, we orient the d^3 slots of our ciphertext into a 3-dimensional cube in xyz space determined by:

1. Set $(0, 0)$ to be the first slot, at the origin
2. Generating a vector of length d along the y -axis by summing existing slots with elements from $\langle (0, d^2/2) \rangle$
3. Generating d vectors of length d along the z -axis by summing existing slots with elements from $\langle (0, d/2) \rangle$
4. Generating d^2 vectors of length d along the x -axis by summing existing slots with elements from $\mathbb{Z}_2 \times \mathbb{Z}_{d/2}$

Example A1 *Let $d = 4$. Then $m = 128$, $\phi(m) = 64$, and $\Omega_{128} \cong \mathbb{Z}_{128}^\times \cong \mathbb{Z}_2 \times \mathbb{Z}_{32}$ with ψ the isomorphism as defined in Theorem 1.*

Our ciphertext elements have structure:

1. *Set the slot $(0, 0)$*
2. *Slots along y -axis: $\langle (0, 8) \rangle$*
3. *Slots along z -axis: $\langle (0, 2) \rangle$*
4. *Slots along y -axis determined by: $(0, 0), (0, 1), (1, 0), (1, 1)$*

Then row swaps are by $(0, 8)$ and square swaps $(0, 2)$.